

PropManage — Viva Examiner's Explanation Guide

Slide-by-slide presentation script (45-60 seconds each) plus likely examiner questions with model answers, for confident, natural delivery during project review, seminar, or viva.

Slide 1 — Title

This is the title slide for my project, "Property Management Application using Power BI." It states the project name, my name, my guide Dr. P. Kayal from the Department of Information Technology, and the college, BVRIT Hyderabad College of Engineering for Women. The title signals two things at once: this is a full property management system, and it has a dedicated business-intelligence component built on Power BI. I open here to set expectations that this is a working, implemented system rather than a purely theoretical proposal. The guide's name establishes the academic supervision behind the work. It takes only a few seconds, but it is my first impression, so I say it clearly and confidently before moving into the actual content.

Possible Viva Questions:

1. Why did you choose this particular title?

— It reflects the two pillars of the system — full property lifecycle management and a dedicated Power BI analytics layer — which together distinguish it from a plain listing app.

2. What was your individual role in this project?

— State this based on your actual contribution — for example, handling the full stack: design, backend, frontend, security, and testing.

3. Why did you pick this domain, property management, specifically?

— It's a sector that has lagged behind banking and e-commerce in digital adoption, so there was a clear, real gap to address with a unified digital platform.

Slide 2 — Table of Content

This slide gives the panel a roadmap of everything I will cover, organized into eighteen sections across two columns. I move from the problem and motivation, through literature review and research gaps, into system design and architecture, then algorithms and modules, and finally results, execution, and conclusion. I summarize it verbally as five movements: problem, related work, design, results, and conclusion, rather than reading every line. This structure mirrors standard academic project reporting, moving from why the project matters to how it was built to what it achieved. Having this map up front also means that if the panel wants to jump to a specific topic later, they already know where it sits. It is a short slide, so I move through it briskly and use it mainly to orient the audience.

Possible Viva Questions:

1. Why did you order the sections this way?

— It follows the standard research-to-implementation flow: motivate the problem, ground it in literature, present the design, then prove it works with results.

2. Which section carries the most weight in your evaluation?

— System Architecture and Result Analysis together, since one explains how the system is built and the other proves it performs as claimed.

Slide 3 — Abstract

The abstract summarizes my entire project in five sentences. PropManage is a MERN-stack platform — MongoDB, Express.js, React.js, and Node.js — built to unify property cataloguing, visit booking, payment handling, contract management, and maintenance tracking into one system. The problem it solves is that small and mid-size property operators today rely on fragmented manual workflows like spreadsheets and phone calls. My solution adds three things that don't normally come together: cryptographically verified payments, real-time staff notifications, and a multilingual voice-enabled AI assistant, backed by a Power BI analytics layer. I close by stating my two headline results — 38 out of 38 functional tests passed, and 14 out of 14 security tests passed — so the panel has concrete proof in mind from the start. Every claim here is expanded and justified later.

Possible Viva Questions:

1. In one line, what problem does your project solve?

— It replaces fragmented, manual property-management workflows with one secure, real-time, AI-assisted platform.

2. What makes this different from a generic property listing app?

— The combination of verified payments, real-time staff alerts, and a multilingual voice assistant together — not any single feature alone.

3. Why the MERN stack specifically?

— It unifies frontend, backend, and database around JavaScript and JSON, reducing the data-transformation overhead of managing multiple languages across layers.

Slide 4 — Introduction

This slide sets the background: property seekers and small or mid-size operators still coordinate through phone calls, messaging apps, and paper records rather than a digital system. My motivation comes from contrasting this with banking, hospitality, and e-commerce, sectors that digitized years ago, which highlights a specific and well-documented lag in real estate. The importance of solving this is that a unified, secure, AI- and voice-assisted platform reduces administrative overhead and, specifically, improves accessibility for users who are not comfortable in English. I also list real-world applications — rental and sale management firms, individual property owners, brokers, and tenant or buyer self-service portals — to show the target market is broad. This slide bridges naturally from the general problem into the more specific, evidence-backed Problem Statement on the next slide.

Possible Viva Questions:

1. Why is real estate specifically lagging in digital adoption?

— Small and mid-size operators often lack the budget for enterprise-grade systems, so they default to manual coordination even as other industries digitized.

2. Who benefits most from this system?

— Small and mid-size property operators and their tenants or buyers, especially non-English speakers who gain voice-based accessibility.

Slide 5 — Problem Statement

This slide states the core problem directly: no single system today covers the full property lifecycle, from listing to maintenance tracking, for small and mid-size operators. Manual coordination leads to four concrete failures — double-booked property visits, payment records that cannot be independently verified, delayed staff awareness of

new bookings or payments, and no assistance for users outside working hours or fluent only in a regional language. I add a fifth gap: no live, self-service business-intelligence view for owners or admins, which directly motivates the Power BI component introduced later. Each of these problems maps to a specific feature I built, so nothing on this slide is rhetorical. The phrase "cannot be independently verified" for payments is worth remembering, because it is the exact language I justify technically a few slides later with the HMAC-SHA256 payment verification algorithm.

Possible Viva Questions:

1. Why is double-booking such a serious problem?

— It wastes staff time and damages trust with clients; a real-time booking and notification system prevents staff from missing overlapping requests.

2. How does your project specifically address the payment verification problem?

— By recomputing an HMAC-SHA256 signature on the server using the order and payment IDs, so a payment record is only accepted if it cryptographically matches what the gateway actually processed.

3. Isn't this problem specific to only one type of operator?

— No — the same fragmentation affects both rental and purchase transactions, and both individual owners and small management firms, which is why the platform is built to serve all of them.

Slide 6 — Objectives

This slide lists seven concrete, measurable objectives rather than vague goals, and each one is proven met later on the Result Analysis slide. They group into three themes: unifying the property lifecycle into one platform, enforcing security and access control, and adding accessibility through voice AI and Power BI analytics. I specify a concrete performance target — sub-second notification latency — rather than just saying "fast," so the claim is falsifiable and checked later. The access-control objective is precise: a de-activated account must lose access on its very next request, not merely when its token eventually expires, which matters technically. I also name the multilingual scope explicitly as English, Hindi, and Telugu, rather than an open-ended claim. The final objective, empirical evaluation, frames the whole second half of my presentation as proof rather than description.

Possible Viva Questions:

1. Which objective was the most technically challenging to achieve?

— Cryptographic payment verification, because it required understanding exactly which data the gateway signs and rejecting any client-supplied data that wasn't independently verifiable.

2. Why sub-second latency specifically as a target?

— It removes the polling delay of manual, inbox-based workflows entirely, so staff are notified as fast as the network allows rather than on a fixed checking interval.

3. Were all seven objectives fully achieved?

— Yes — each is directly matched to a measured result in the Result Analysis slide, such as the 38/38 functional pass rate and sub-500ms latency at 100 concurrent users.

Slide 7 — Scope of Work

This slide draws a clear line between what is in scope and what is out of scope for this project phase. In scope covers property listing and search, visit and purchase booking, UPI and Razorpay payments, rental and purchase contracts, maintenance tickets, real-time admin notifications, the multilingual voice assistant, and Power BI analytics. Out of scope, deliberately, are a native mobile app, full interface localization with right-to-left layout, IoT or smart-lock

integration, and predictive pricing analytics — and all four of these reappear later as named Future Scope items, showing the boundary was planned rather than accidental. I also name the three target user groups: property seekers and tenants, property owners, and platform administrators, which map directly onto the system's dual-portal design. This slide gives me a ready, confident answer for any "why didn't you build X" question during the viva.

Possible Viva Questions:

1. Why exclude a mobile app from this phase?

— The REST API is already mobile-ready; a React Native client is explicitly future work and would reuse the same backend without any redesign.

2. How did you decide what was in scope versus out of scope?

— By prioritizing features that directly closed the problems identified earlier — lifecycle coverage, verified payments, real-time alerts, and accessibility — while parking larger infrastructure investments like IoT for later.

Slide 8 — Literature Review

This slide presents six papers in a structured comparison table covering the paper title and authors, the journal, the methodology, the key finding, and its limitation. The six span four themes that match my project's technical pillars: MERN architecture, LLM chatbots, multilingual voice assistants, and JWT security auditing, plus a systematic review of AI adoption specifically in property management. Every source is real, recently published between 2023 and 2025, and drawn from reputable venues — nothing here is fabricated to pad the table. Each paper's limitation column is deliberately the seed of a research gap I state on the very next slide, so this table is evidence, not decoration. For instance, one paper identifies multilingual voice-assistant barriers conceptually but never builds a working system — which is precisely the gap PropManage fills. I keep this table to six focused rows rather than twenty generic ones, prioritizing relevance over sheer volume.

Possible Viva Questions:

1. Why only six papers instead of a longer list?

— Quality and direct relevance to the four technical pillars of this project mattered more than a large but generic reference count.

2. What is the common limitation across all six papers?

— Each addresses one piece — architecture, chatbots, voice, or security — in isolation, but none integrates them into one deployed, evaluated property-management system.

3. How did you select these particular papers?

— Each maps directly onto a specific design decision in the system, so they function as justification for real choices, not generic background reading.

Slide 9 — Research Gaps

This slide condenses the literature review into exactly three gaps, which is deliberately a small, memorable number rather than a long list. First, architecture and persistence choices like MERN and MongoDB are evaluated in isolation in prior work, never inside a real multi-entity, transaction-heavy system like this one. Second, security mechanisms such as JWT auditing are studied as standalone diagnostic tools, not as integrated parts of one adversarially tested deployed system. Third, and most central to my contribution, no reviewed system combines domain-grounded, multilingual, voice-enabled conversational assistance with verified payments and real-time notification in one empirically evaluated platform. I explicitly state that PropManage addresses all three gaps at once, which is the

single strongest sentence in my whole literature section, and I deliver it slowly. Everything before this slide is motivation; everything after it is the proposed solution and its proof.

Possible Viva Questions:

1. What is genuinely novel here, versus just combining existing APIs?

— The novelty is the integration itself and the specific engineering choice of a zero-cost, browser-native voice interface layered onto a live-data-grounded chatbot — not any one library in isolation.

2. How do you know these gaps are real and not just convenient for your project?

— Each gap is drawn directly from a stated limitation in a real, cited paper on the Literature Review slide, not asserted independently.

3. Could gap three be closed by simply using an existing chatbot platform?

— Off-the-shelf chatbot platforms don't provide live, domain-specific data grounding combined with zero-cost multilingual voice input and output in one package, which is exactly the combination this project builds.

Slide 10 — Existing Systems

This slide categorizes today's alternatives into three types: fully manual coordination, browsing-only listing portals like Bayut.com or Dubizzle, and enterprise commercial suites like Buildium or AppFolio. Even the enterprise suites, despite covering more of the property lifecycle, still lack verified real-time payments and AI or voice assistance, so this is a feature-based comparison, not simply expensive versus cheap. I list five disadvantages of the current landscape: manual and time-consuming coordination, unverifiable payments, delayed staff notification, no conversational or voice assistance, and high cost with poor scalability for smaller operators. These same four platforms reappear later in my feature comparison table, keeping the argument consistent throughout the presentation. This slide sets up a direct, one-for-one contrast with the Proposed System slide immediately after it, where each weakness here gets answered.

Possible Viva Questions:

1. Have you actually tested Buildium or AppFolio yourself to confirm these gaps?

— This comparison is based on published feature documentation for these platforms rather than hands-on enterprise trials — it's a feature-level literature comparison.

2. Why do listing portals like Bayut not already solve this problem?

— They are browsing-only — they don't support booking workflows, payment verification, contract management, or real-time staff notification at all.

Slide 11 — Proposed System

This slide frames my new workflow as a single, role-segmented, dual-portal MERN platform, directly answering the fragmentation problem from earlier. The new features are real-time WebSocket notification, HMAC-SHA256 verified payments, a multilingual voice AI assistant, and Power BI analytics, presented as one integrated system rather than separate add-ons. I claim four advantages — faster, more secure, more accessible, and more scalable — and each one is backed by a specific mechanism I explain in later slides, not just an adjective. The novelty statement here is the one sentence I say most carefully: a browser-native, zero-cost multilingual voice assistant integrated with a live-data-grounded chatbot, a combination not found together in any reviewed system. This slide functions as an executive summary before the technical depth increases, and it directly answers, point for point, the five weaknesses named on the previous slide.

Possible Viva Questions:

1. What exactly makes this system 'more secure' than existing tools?

— Payment integrity is cryptographically verified server-side rather than trusted from the client, and role-based access is re-checked on every request rather than relying on a token's validity alone.

2. How is the voice assistant 'zero-cost'?

— It uses the browser's built-in Web Speech API for both speech recognition and speech synthesis, so there is no separate paid speech-processing service or extra infrastructure required.

3. Could this system scale to a much larger number of properties?

— The architecture supports it structurally, though the current single parallelized query batch behind the admin dashboard is one known limitation, which is why fault-tolerant aggregation is listed under Future Scope.

Slide 12 — System Architecture

This diagram shows the real request flow through the system: from the User, to the Frontend built in React, to Authentication using JWT and role-based access control, to the Voice Assistant powered by Gemini and the Web Speech API, then to the Backend APIs in Node.js and Express, then to the MongoDB Database, and finally out to the Admin Panel and Power BI Reports and Analytics. I deliberately place Authentication before the Voice Assistant box to show that even conversational or voice requests are authorized like any other protected action, not treated as a special, unauthenticated shortcut. Both the client-user and client-admin portals converge on the same backend and database, reflecting the real dual-portal, shared-backend design. This is the single diagram I return to if I get stuck on any later question, since it re-orientes the whole system in one glance.

Possible Viva Questions:

1. Why is the voice assistant a separate box instead of being part of Backend APIs?

— It's drawn separately because it's a distinct cross-cutting concern — a client-side speech layer plus a server-mediated Gemini call — not because it runs on a separate physical server; architecturally it's still part of the same Node.js application tier.

2. Why MongoDB instead of a relational database like MySQL?

— The domain has many interlinked but loosely structured entities — properties, bookings, contracts — that suit a document model well, and MERN's single JSON representation across all tiers reduces cross-layer data-transformation code.

3. What happens if the database connection fails?

— The system uses Mongoose's reconnection logic with backoff to recover automatically; full high-availability failover across multiple database nodes is noted as future work, not hidden as a gap.

Slide 13 — Methodology / Algorithms

This slide documents five core mechanisms that do the real technical work, each explained through its purpose, its working, and its advantage. JWT authentication re-checks a user's role and active status on every single request, so a blocked account loses access immediately rather than only once its token eventually expires. HMAC-SHA256 payment verification is my strongest security claim: the server independently recomputes the expected signature from the order and payment IDs and rejects any mismatch before touching a financial record. Real-time WebSocket notification broadcasts events to the admin room immediately after the database saves them, with no polling delay at all. The multilingual voice pipeline traces microphone input through browser speech recognition, into a language-conditioned Gemini prompt grounded in live data, back through a streamed reply and speech synthesis. Finally, the

Power BI pipeline aggregates data inside MongoDB itself, so it scales with reporting complexity rather than the number of records.

Possible Viva Questions:

1. Why recompute the HMAC signature instead of just trusting the payment status returned by the gateway?

— A status field can be reported directly by the client and isn't cryptographically bound to anything; recomputing the signature server-side proves the data wasn't tampered with in transit.

2. Why Gemini specifically, and not GPT or Claude?

— Gemini was the provider actually integrated and load-tested in this build; the prompt-assembly layer is provider-agnostic, so switching providers would not require redesigning the pipeline.

3. Is your voice assistant using true Retrieval-Augmented Generation, or RAG?

— Not yet — today it uses a lighter-weight context-assembly approach where a live database snapshot is injected directly into the prompt; migrating to embedding-based vector retrieval is named explicitly as future work.

Slide 14 — Working Modules

This slide presents seven functional modules as a clean visual grid: User, Authentication, Voice Assistant, Booking and Payment, Admin, Database, and Reporting. It mirrors the System Architecture diagram but reframes the same system as ownership boundaries — who is responsible for what — rather than a step-by-step request flow. Booking and Payment are grouped into a single module because, in the real implementation, they're tightly coupled — a booking almost always carries an associated payment record. I keep the Voice Assistant as its own distinct module rather than folding it into Authentication, to underscore that it is a genuine, named capability of the system, not a minor add-on. This slide gives the panel a simple mental model for asking any implementation question, such as which module handles a particular concern, and I move through it briskly since each module was already explained in more depth earlier.

Possible Viva Questions:

1. Why combine Booking and Payment into a single module rather than keeping them separate?

— Because in the actual data model a booking record and its payment status are directly linked, so treating them as one cohesive module reflects the real implementation.

2. Which module was hardest to build?

— The Voice Assistant module, since it had to coordinate browser speech APIs, language selection, and a grounded, streamed AI reply all in one coherent flow.

Slide 15 — Requirements

This slide splits requirements into Hardware and Software using the same visual style as my Technology Stack slide for consistency. On the hardware side, requirements are modest: a dual-core processor at 2 gigahertz or above, 4 gigabytes of RAM minimum with 8 recommended, 500 megabytes of storage, and a stable internet connection. On the software side, the system supports Windows, macOS, and Linux, and runs on Node.js 18 LTS and React.js 18.2, alongside MongoDB Atlas, VS Code, and the Razorpay, Google Gemini, Cloudinary, and Mailjet APIs, plus Power BI Desktop or Service for analytics. This reinforces a claim I make in both my Abstract and Conclusion — that the system is accessible without an enterprise budget — since every listed requirement is either open-source or has a usable free tier. This slide is short by design; it exists to satisfy a standard checklist, not to carry argumentative weight.

Possible Viva Questions:

1. Are any of these requirements paid or licensed software?

— No — every tool and API listed is either fully open-source or has a free tier sufficient for this system's current scale.

2. Would this run on a low-end laptop?

— Yes, the stated requirements are intentionally modest — a standard dual-core machine with 4GB of RAM is sufficient for development and testing.

Slide 16 — Technology Stack

This slide organizes the full technology stack into two tables: Frontend and Backend, and Third-Party Services and AI. The first covers React.js 18 for both portals, Node.js and Express.js for the REST API, MongoDB Atlas and Mongoose for persistence, and Socket.IO for the real-time layer — the four pillars of my MERN implementation. The second covers Razorpay with HMAC-SHA256 for payments, the Gemini API together with the Web Speech API for AI and voice, Cloudinary and Mailjet for storage and email, and Power BI's REST API for analytics. Each row states not just the technology but its purpose, so I can answer any "why this tool" question by pointing directly at that column. This entire table replaced the original presentation template's sample data-science package list, which was completely unrelated to this project, with the real stack actually used in PropManage. It's my natural go-to slide whenever a technology-choice question comes up.

Possible Viva Questions:

1. Why Razorpay specifically as the payment gateway?

— It's a widely used gateway with strong signature-verification documentation and INR support relevant to the target market, and it was the gateway actually integrated and tested in this build.

2. Why Socket.IO instead of plain WebSockets?

— Socket.IO adds automatic fallback to HTTP polling when a WebSocket connection isn't available, making the real-time layer more resilient across different network conditions.

3. Could this stack be swapped for a different one, like Django or Spring Boot?

— Structurally yes, but MERN was chosen specifically because a single JSON representation flows across the database, API, and both client portals, minimizing cross-layer transformation code for this data-heavy domain.

Slide 17 — Experimental Results

This slide reports the real dataset scale used for testing: 45 properties, 120 users, 280 bookings, 195 payments, 41 rental and 12 purchase contracts, and 310 financial plus 87 service transactions. The two headline pass rates recur throughout my presentation: 38 out of 38 functional test scenarios passed, and 14 out of 14 adversarial security checks passed. I also state the concurrent-load result — 100 users sustained at sub-500-millisecond mean latency with under 0.2 percent errors — which is my strongest performance claim. The bar chart shows endpoint latency across six operations, and the AI Chat endpoint is visibly the slowest at 2080 milliseconds; I explain that this reflects a genuine round-trip to an external AI provider, not an inefficiency in my own code. Every number here comes from a real, populated test database and a scripted test suite, not a tiny, unrepresentative demo.

Possible Viva Questions:

1. Was this tested with real users or synthetic data?

— Synthetic but realistically scaled data, seeded specifically to reflect production-level query complexity, combined with scripted adversarial test cases rather than live production traffic.

2. Why is the AI Chat endpoint so much slower than the others?

— It requires a round-trip to an external large language model API for a streamed conversational response, which is normal for chat-style interaction and not on a page-load-critical path.

3. How many total test scenarios did you run altogether?

— 38 functional test scenarios plus 14 separate adversarial security test cases, all of which passed.

Slide 18 — Execution

This slide walks through the live system using four labelled screens: Login, Dashboard, Voice Assistant, and Admin Panel. I narrate them as one connected user journey — logging in, viewing the dashboard, interacting with the voice assistant, and then switching to the admin's perspective — rather than describing four disconnected screenshots. If real screenshots are pasted in by viva day, I speak naturally about what each screen does and why it matters, rather than reading captions. This is often the most persuasive part of a project viva, because it demonstrates hands-on familiarity with an actually running system rather than only slides. I keep my narration brief per screen and let the interface itself carry the evidence. If a screenshot is still a placeholder, I state that plainly and move on without over-apologizing.

Possible Viva Questions:

1. Can you demonstrate the voice assistant live right now?

— Yes — walk through selecting a language, speaking a query, and showing the transcribed text and spoken reply, exactly as implemented with the Web Speech API and Gemini.

2. What happens if the browser doesn't support speech recognition?

— The microphone control is disabled with a clear explanatory message rather than failing silently — this was specifically tested for browsers like Firefox that lack full support.

Slide 19 — Result Analysis

This slide consolidates every headline metric into one table: functional correctness, adversarial security tests, mean latency, two error-rate figures, and real-time notification delivery time. I restate the 38 out of 38 and 14 out of 14 results as explicit 100 percent pass rates, always paired with their exact denominators so the claim reads as precise evidence, not an inflated number. I report mean latency of 494 milliseconds alongside its 95th-percentile figure of 743 milliseconds at 100 concurrent users, giving both a typical and a worst-case view. The field-observed notification delivery time of under two seconds is separate, qualitative evidence from actual usage, distinct from the scripted load tests above it. This slide is my evidentiary centerpiece — every objective stated earlier has a matching number here, closing the loop the whole presentation opened with.

Possible Viva Questions:

1. Does 100% security pass rate mean the system is completely secure?

— It means all fourteen specific adversarial checks that were designed and tested passed — evidence against those particular threat categories, not a claim of exhaustive security coverage against every possible attack.

2. Why report both mean and 95th-percentile latency?

— The mean shows typical performance, while the 95th percentile shows how the slower requests behave, giving a fuller and more honest picture of real-world responsiveness.

3. What would happen to these numbers at 500 concurrent users?

- Based on the load-test trend, error rates rise noticeably beyond 150 users, so horizontal scaling of the server and MongoDB Atlas auto-scaling would be needed to sustain the same latency at that scale.

Slide 20 — Conclusion

I open my conclusion by restating what was delivered in one sentence: a dual-portal, three-tier MERN platform unifying cataloguing, booking, payments, contracts, and maintenance. I explicitly confirm all five objectives were met — real-time notification, HMAC-verified payments, role-based access control, multilingual voice AI, and Power BI analytics. I restate the empirical validation numbers one final time, since repetition helps the panel remember the strongest evidence: 38 out of 38, 14 out of 14, and 100 concurrent users at sub-500-millisecond latency. I state the practical benefit in plain business terms — reduced administrative overhead, improved payment trust, and voice accessibility in English, Hindi, and Telugu. I close with the broader argument that this proves a production-quality system can be built entirely on free-tier tooling, which is a message I deliver slightly slower and with more confidence, since it's my landing point before Future Scope.

Possible Viva Questions:

1. What is the single most important achievement of this project?

- Proving that a small operator can get enterprise-grade capabilities — verified payments, real-time alerts, and AI/voice accessibility — entirely on free-tier tooling.

2. Did you meet every objective you set out to achieve?

- Yes, all five design objectives were met and are each backed by a specific measured result in the Result Analysis slide.

Slide 21 — Future Scope

This slide lists six concrete future directions, each grounded in something already built rather than a vague aspiration. Migrating the AI's grounding to full retrieval-augmented generation using MongoDB's vector search would extend, not replace, today's context-assembly approach. A React Native mobile app is feasible because it would reuse the exact same REST API and business logic without any redesign. Full interface localization and right-to-left layout would extend today's voice-level multilingual support to the interface itself. Predictive analytics would forecast rental pricing, vacancy, and late-payment risk from data the system already collects, moving analytics from descriptive to prescriptive. Fault-tolerant analytics aggregation directly answers the one architectural limitation I admit honestly earlier, and IoT integration would add smart locks and meters for keyless access and automated readings.

Possible Viva Questions:

1. Which of these would you personally prioritize if you had one more month?

- Predictive analytics or the RAG migration — both build directly on data and infrastructure the system already has, rather than requiring new external integrations.

2. Why wasn't retrieval-augmented generation implemented in this phase?

- The current context-assembly approach was sufficient and lower-risk at this catalogue scale; full RAG becomes valuable specifically as the property catalogue grows larger.

Slide 22 — References

This slide lists the first eight of fifteen total references in full IEEE-style citation format, covering authors, title, journal or venue, and year. It includes the foundational security references that justify my authentication and payment-verification design, the architecture references that justify my technology stack choice, the AI and chatbot references that justify my conversational-assistant design and its known limitations, and the real-estate digital-transformation reference that anchors my project's core motivation. I don't read citations aloud in the viva — I simply state the range and venues in one sentence, as I'm doing now. Every single reference is real, verifiable, and recently published, drawn from reputable journals and conferences rather than invented to inflate the citation count, which matters for academic credibility.

Possible Viva Questions:

1. Are all these references peer-reviewed?

— They are drawn from reputable indexed venues including journals and conference proceedings from IEEE-adjacent, Springer, and MDPI sources published between 2023 and 2026.

2. Which reference most directly supports your security design?

— The JWTKey study by Bowen et al., which found exploitable cryptographic weaknesses across a majority of audited real-world JWT implementations.

Slide 23 — References (contd.)

This slide continues and completes the reference list from entry nine through fifteen. It includes the property-management-specific AI adoption study that supports my Research Gap claim that deployed, evaluated AI-integrated property systems are still scarce in the literature. It includes both retrieval-augmented-generation references that justify the specific future-work direction I named earlier, and the MERN performance-optimization reference that supports the specific engineering techniques mentioned in my Algorithms and Results discussion. It also includes the two references most directly tied to this project's core novelty — one on multilingual voice-assistant barriers, and one demonstrating a working browser-based multi-language voice assistant. I close by moving past this slide quickly, saying simply, "and the remainder of the reference list," since its purpose is completeness, not further discussion.

Possible Viva Questions:

1. Which reference is most directly tied to your multilingual voice assistant?

— Sabharwal and Sahni's study on the barriers voice assistants face in Hindi and other Indian languages, alongside Pinniboyina's working browser-based multi-language voice assistant demonstration.

2. Why include a healthcare chatbot reference in a property management project?

— The Khamaj study demonstrates that adding voice input and output measurably improves usability for users underserved by text-only interfaces — the same accessibility argument applies directly here.